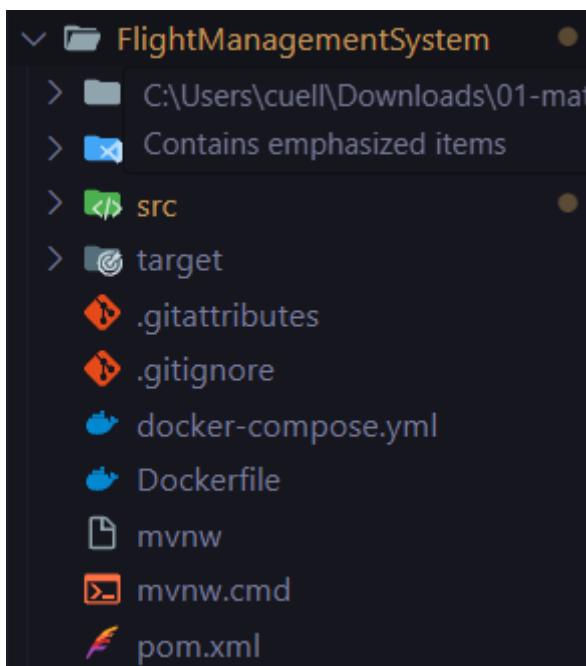


Guía práctica: CI/CD con Docker

1. Requisitos iniciales

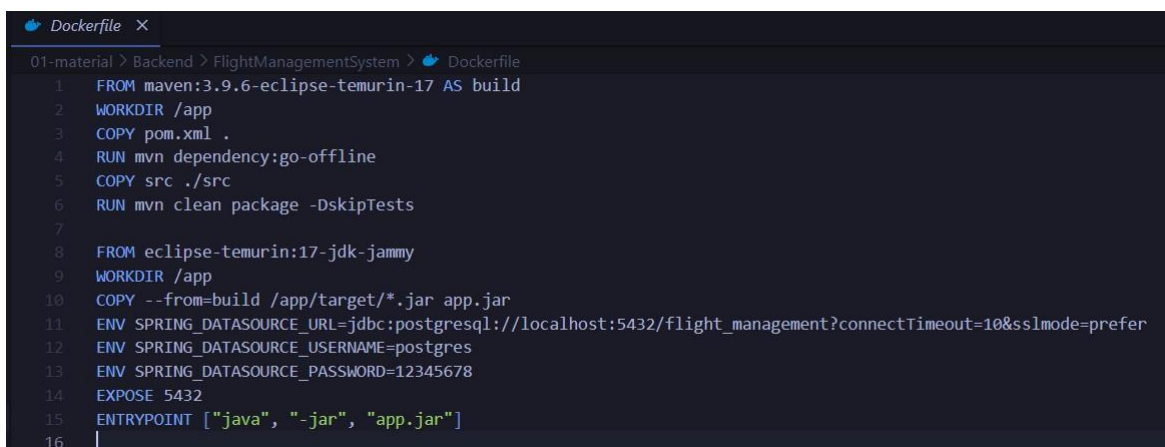
Antes de comenzar asegúrate de contar con: - Docker Desktop instalado y en ejecución (Windows / Mac / Linux). - Git y un editor de código de tu preferencia.

2. Organización del proyecto



Se recomienda colocar todos los archivos de configuración en la carpeta raíz del proyecto (donde normalmente está pom.xml en caso de proyectos Java).

3. Dockerfile



En la raíz del proyecto crea un archivo llamado Dockerfile. Este archivo se encargará de: 1.

Compilar tu aplicación .jar usando una imagen de Maven (evitando que necesites Maven local). 2. Generar una imagen ligera con un JRE para ejecutar la aplicación.

4. docker-compose.yml (Backend + Base de Datos)

```
1  services:
2    postgres:
3      container_name: serve-postgres-sena
4      image: postgres:16
5      restart: always
6      environment:
7        POSTGRES_PASSWORD: n7cuellar
8        POSTGRES_USER: postgres
9        POSTGRES_DB: flight_management
10     ports:
11       - "5432:5432"
12     networks:
13       - network_local_server
14     volumes:
15       - postgres_data:/var/lib/postgresql/data
16       - "/home/ubuntu/db_backups:/backups/postgres"
17     healthcheck:
18       test: ["CMD-SHELL", "pg_isready -U postgres"]
19       interval: 30s
20       timeout: 10s
21       retries: 5
22     labels:
23       - com.sena.group=databases
24
25 1  backend:
26   2     container_name: backend-flight-management
27   3     build:
28   4       context: .
29   5       dockerfile: Dockerfile
30   6     restart: always
31   7     depends_on:
32   8       postgres:
33   9         condition: service_healthy
34  10    environment:
35  11      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:54
36  12      SPRING_DATASOURCE_USERNAME: postgres
37  13      SPRING_DATASOURCE_PASSWORD: n7cuellar
38  14    ports:
39  15      - "8081:8081"
40  16    networks:
41  17      - network_local_server
42  18
43  19    volumes:
44  20      postgres_data:
45  21        driver: local
46  22
47  23    networks:
48  24      network_local_server:
49  25        driver: bridge
50  26        name: network_local_server
```

Crea el archivo docker-compose.yml, que permitirá levantar varios servicios de manera conjunta. - db → inicia PostgreSQL con usuario, contraseña y una base de datos inicial. - app → construye y ejecuta el backend a partir del Dockerfile. - depends_on → asegura que la base de datos arranque antes de la aplicación. - volumes → mantiene los datos persistentes aunque los contenedores se

detengan o reinicien.

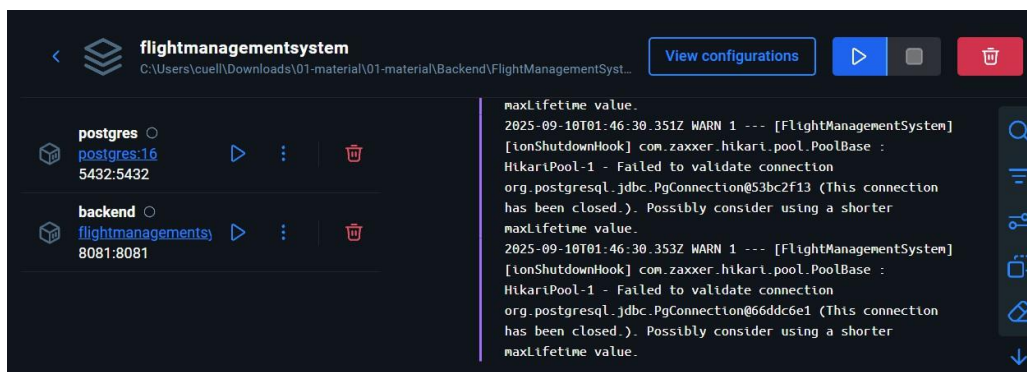
5. Construcción y despliegue

```
PS C:\Users\cuell\Downloads\01-material\01-material\Backend\FlightManagementsystem> docker-compose up -d --build
Compose can now delegate builds to bake for better performance.
To do so, set COMPOSE_BAKE=true.
[+] Building 279.2s (18/18) FINISHED
```

```
[+] Running 5/5
✓ backend Built
=> => unpacking to docker.io/library/flightmanagementsystem-backend:latest
=> [backend] resolving provenance for metadata file
[+] Running 5/5
✓ backend Built 0.0s
✓ Network network_local_server Created 0.2s
✓ Volume "flightmanagementsystem_postgres_data" Created
✓ Container serve-postgres-sena Healthy
✓ Container backend-flight-management Started
```

Desde la carpeta donde se encuentre el archivo docker-compose.yml, ejecuta: `--build` → fuerza la reconstrucción de la imagen del backend. `-d` → corre los servicios en segundo plano (modo detached). Cuando termine el proceso, verás un mensaje de confirmación en la terminal.

6. Visualizar en Docker Desktop



Abre Docker Desktop para comprobar que los contenedores estén levantados y corriendo correctamente.

7. Crear esquemas en la base de datos

```

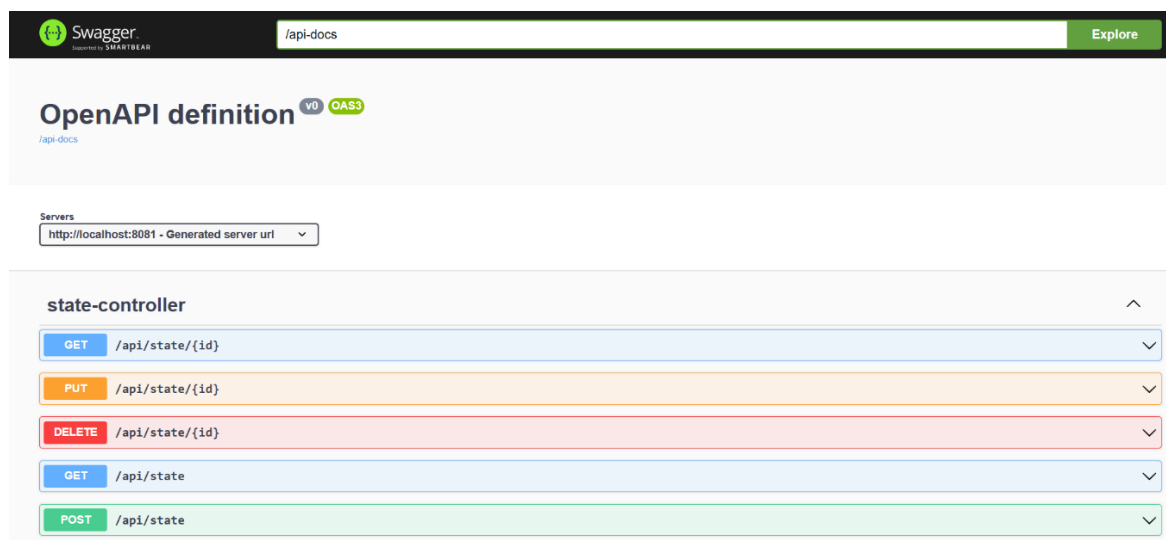
PS C:\Users\cuell> docker exec -it serve-postgres-sena psql -U postgres -d flight_management
psql (16.10 (Debian 16.10-1.pgdg13+1))
Type "help" for help.

flight_management=# CREATE SCHEMA aircraft_management;
CREATE SCHEMA flight;
CREATE SCHEMA geolocation;
CREATE SCHEMA human_resources;
CREATE SCHEMA infrastructure;
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA

```

Usa la terminal para ejecutar los comandos que generen los esquemas necesarios en tu base PostgreSQL.

8. Probar con Swagger



Finalmente, accede a Swagger y valida que los endpoints del backend respondan como se espera.

Con esto tendrás un flujo funcional de despliegue con Docker y docker-compose, listo para desarrollo y pruebas.